

```
CREATE TABLE Fornitori (
  fid INT PRIMARY KEY,
  fnome VARCHAR(50) NOT NULL,
  indirizzo VARCHAR(100)
);
```

```
CREATE TABLE Pezzi (
  pid INT PRIMARY KEY,
  pnome VARCHAR(50) NOT NULL,
  colore VARCHAR(20),
  peso DECIMAL(5,2)
);
```

```
CREATE TABLE Catalogo (
  fid INT,
  pid INT,
  costo DECIMAL(8,2),
  PRIMARY KEY (fid, pid),
  FOREIGN KEY (fid) REFERENCES Fornitori(fid),
  FOREIGN KEY (pid) REFERENCES Pezzi(pid)
);
```

Per ogni fornitore
calcolare quanti pezzi esso
fornisce e il prezzo medio

DALTA SCANSATA
LEZIONE

▪ **Esempio:**
SELECT DNO, COUNT(*), AVG(SALARY)
FROM EMPLOYEE
GROUP BY DNO;

↳ lo stesso ma col nome
nel risultato

```
SELECT F.fnome AS Fornitore, COUNT(pid) AS numPezzi,
  avg(costo) AS costoMedio
FROM Fornitori AS F JOIN Catalogo AS C ON F.fid=C.fid
GROUP BY F.fnome;
```

Fornitore	numPezzi	costoMedio
1 Polito	2	0.085
2 RomaTre	3	0.09333333333333333
3 Sapienza	3	0.11
4 Tor Vergata	2	0.185

```
SELECT fid, COUNT(pid), avg(costo)
FROM Catalogo
GROUP BY fid;
```

	fid	COUNT(pid)	avg(costo)
1	1	3	0.11
2	2	2	0.185
3	3	3	0.09333333333333333
4	4	2	0.085

Calcolare il costo totale di catalogo per ogni colore di pezzo,
ordinando per costo crescente.

```
SELECT p.colore, SUM(c.costo) AS costo_totale
FROM CATALOGO c JOIN PEZZI p ON c.pid = p.pid
GROUP BY p.colore
ORDER BY costo_totale;
```

colore	costo_totale
1 blu	0.25
2 verde	0.4
3 rosso	0.5

Trovare i fornitori il cui costo medio dei pezzi sia
superiore a 0.10, usando HAVING

```
SELECT f.fid, f.fnome, AVG(c.costo) AS costo_medio
FROM FORNITORI f JOIN CATALOGO c
  ON f.fid = c.fid
GROUP BY f.fid, f.fnome
HAVING AVG(c.costo) > 0.1;
```

fid	fnome	costo_medio
1	1 Sapienza	0.11
2	2 Tor Vergata	0.185

Elencare i fornitori che forniscono almeno un pezzo che non e' di colore 'rosso',
usando LEFT JOIN.

↳ uso didattico del LEFT JOIN

```
SELECT DISTINCT f.fid, f.fnome
FROM FORNITORI f
  JOIN CATALOGO c
    ON f.fid = c.fid
  LEFT JOIN PEZZI p
    ON c.pid = p.pid AND p.colore = 'rosso'
WHERE p.pid IS NULL;
```

Questa soluzione col LEFT JOIN è puramente didattica
perché la soluzione naturale è un'altra.

```
SELECT *
FROM Fornitori AS F JOIN Catalogo AS C ON F.fid=C.fid
  JOIN Pezzi AS P ON P.pid=C.pid
WHERE P.colore <> 'rosso';
```

1

```
SELECT *
FROM Fornitori AS F
JOIN Catalogo AS C ON F.fid=C.fid
LEFT JOIN Pezzi AS P ON P.pid=C.pid AND P.colore='rosso';
```

	fid	fnome	indirizzo	fid	pid	costo	pid	pnome	colore	peso
1	1	Sapienza	Via Cavour, 18	1	10	0.1	10	Vite M4	rosso	0.05
2	1	Sapienza	Via Cavour, 18	1	11	0.15	NULL	NULL	NULL	NULL
3	1	Sapienza	Via Cavour, 18	1	12	0.08	12	Dado M6	rosso	0.03
4	2	Tor Vergata	Via Columbia, 2	2	10	0.12	10	Vite M4	rosso	0.05
5	2	Tor Vergata	Via Columbia, 2	2	13	0.25	NULL	NULL	NULL	NULL
6	3	RomaTre	Viale Ostiense, 159	3	11	0.14	NULL	NULL	NULL	NULL
7	3	RomaTre	Viale Ostiense, 159	3	12	0.09	12	Dado M6	rosso	0.03
8	3	RomaTre	Viale Ostiense, 159	3	14	0.05	NULL	NULL	NULL	NULL
9	4	Polito	Corso Duca degli Abruzzi, 24	4	10	0.11	10	Vite M4	rosso	0.05
10	4	Polito	Corso Duca degli Abruzzi, 24	4	14	0.06	NULL	NULL	NULL	NULL
11	5	Oxford	121 Keble Road	5	13	0.1	NULL	NULL	NULL	NULL
12	6	Cambridge	4 Bridge Street	6	12	0.095	12	Dado M6	rosso	0.03

2

```
SELECT F.fid, F.fnome
FROM Fornitori AS F
JOIN Catalogo AS C ON F.fid=C.fid
LEFT JOIN Pezzi AS P ON P.pid=C.pid AND P.colore='rosso'
WHERE P.pid IS NULL;
```

	fid	fnome
1	1	Sapienza
2	2	Tor Vergata
3	3	RomaTre
4	3	RomaTre
5	4	Polito
6	5	Oxford

Id. e nome dei fornitori che non forniscono alcun pezzo.

```
SELECT F.fid, F.fnome
FROM Fornitori AS F
WHERE F.fid NOT IN (SELECT C.fid FROM Catalogo AS C);
```

Identificativo e nome dei fornitori che non forniscono alcun pezzo rosso.

```
SELECT F.fid, F.fnome
FROM Fornitori AS F
WHERE F.fid NOT IN
  (SELECT DISTINCT F1.fid FROM Fornitori AS F1
   JOIN Catalogo AS C1 ON C1.fid=F1.fid
   JOIN Pezzi AS P1 ON P1.pid=C1.pid
   WHERE P1.colore='rosso')
```

Trovare quanti colori hanno i pezzi forniti da ciascun
fornitore, inclusi i fornitori che non forniscono
alcun pezzo (che avranno 0 colori)

```
SELECT F.fnome, COUNT(DISTINCT P.colore)
FROM Fornitori AS F
LEFT JOIN Catalogo AS C ON C.fid=F.fid
LEFT JOIN Pezzi AS P ON P.pid=C.pid
GROUP BY F.fid;
```

↓
Soluzione equivalente con RIGHT JOIN

```
SELECT f.fid, f.fnome, COUNT(DISTINCT p.colore) AS num_colori
FROM PEZZI p
JOIN CATALOGO c
  ON p.pid = c.pid
RIGHT JOIN FORNITORI f
  ON c.fid = f.fid
GROUP BY f.fid;
```

```
CREATE TABLE Fornitori (
  fid INT PRIMARY KEY,
  fnome VARCHAR(50) NOT NULL,
  indirizzo VARCHAR(100)
);

CREATE TABLE Pezzi (
  pid INT PRIMARY KEY,
  pnome VARCHAR(50) NOT NULL,
  colore VARCHAR(20),
  peso DECIMAL(5,2)
);

CREATE TABLE Catalogo (
  fid INT,
  pid INT,
  costo DECIMAL(8,2),
  PRIMARY KEY (fid, pid),
  FOREIGN KEY (fid) REFERENCES Fornitori(fid),
  FOREIGN KEY (pid) REFERENCES Pezzi(pid)
);
```

AGGIORNAMENTI IN SQL

- In SQL sono previsti tre comandi per modificare il database:
 - **INSERT**
 - **DELETE**
 - **UPDATE**

IL COMANDO INSERT

- Il comando **INSERT INTO** inserisce nuove righe in una relazione.

- Sintassi:

```
INSERT INTO Target [(FieldName,...)]  
VALUES (Value1,...);
```

oppure

```
INSERT INTO Target [(FieldName,...)]  
SELECT FieldName,...  
FROM TableExpression;
```

→ vedi oltre

- Aggiungere una nuova tupla alla relazione **'Employee'**:

```
INSERT INTO EMPLOYEE  
VALUES ('Richard', 'K', 'Marini', '654765876',  
'30-DEC-52', '98 Oak Forest, Katy, TX', 'M', 37000, '987654321', 4);
```

IL COMANDO INSERT (2)

- È possibile non assegnare valori a tutti gli attributi.
- In tal caso, questi avranno il valore di **default** o **NULL**.

- **Esempio:**

```
INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)  
VALUES ('Richard', 'Marini', '654765876');
```

IL COMANDO INSERT - ESEMPIO

- Creare una **tabella temporanea** che ha nome, numero di impiegati e salari totali per ciascun dipartimento:

```
CREATE TABLE DEPTS_INFO ( DEPT_NAME VARCHAR(15),  
                           NO_OF_EMPS INTEGER,  
                           TOTAL_SAL INTEGER);
```

```
INSERT INTO DEPTS_INFO (DEPT_NAME, NO_OF_EMPS, TOTAL_SAL)  
{ SELECT DNAME, COUNT(*), SUM(SALARY)  
  FROM DEPARTMENT, EMPLOYEE  
  WHERE DNUMBER=DNO  
  GROUP BY DNAME;
```

← table expression

- Eventuali aggiornamenti successivi non influenzano la tabella originale. Per aggiornarle, è invece necessario definire una **view**.

USO DI *AUTO_INCREMENT*

- L'attributo **AUTO_INCREMENT** può essere usato per generare un identificatore unico per le nuove righe:

```
CREATE TABLE animals (  
  id INT NOT NULL AUTO_INCREMENT,  
  name VARCHAR(30) NOT NULL,  
  PRIMARY KEY (id)  
) AUTO_INCREMENT=5;
```

a partire da 5

```
INSERT INTO animals (name) VALUES  
( 'dog'), ('cat'), ('penguin'), ('wolf'), ('whale'), ('ostrich');
```

```
SELECT * FROM animals;
```

id	name
5	dog
6	cat
7	penguin
8	wolf
9	whale
10	ostrich

SERIAL

- **AUTO_INCREMENT** non è più supportato da Postgres, il quale invece utilizza **SERIAL** come tipo di dati per identificare un attributo che si auto incrementa per le nuove righe.

```
CREATE TABLE animals (  
  id SERIAL,  
  name VARCHAR(30) NOT NULL,  
  PRIMARY KEY (id)  
);
```

IL COMANDO DELETE

- Il comando **DELETE** rimuove una o più tuple da una relazione.

- Sintassi:

```
DELETE  
FROM TableName  
WHERE Criteria;
```

IL COMANDO DELETE - *ESEMPI*

```
DELETE FROM EMPLOYEE  
WHERE LNAME='Brown';
```

```
DELETE FROM EMPLOYEE  
WHERE DNO IN (SELECT DNUMBER  
              FROM DEPARTMENT  
              WHERE DNAME='Research');
```

IL COMANDO UPDATE

- Il comando **UPDATE** permette di modificare valori in una relazione:

```
UPDATE PROJECT
SET PLOCATION='Bellaire', DNUM=5
WHERE PNUMBER=10;
```

```
UPDATE EMPLOYEE
SET SALARY=SALARY * 1.1
WHERE DNO IN (SELECT DNUMBER
FROM DEPARTMENT
WHERE DNAME='Research');
```

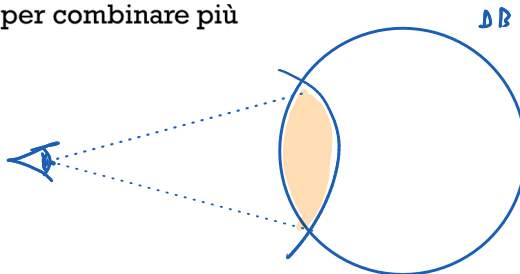
← espress. aritmetica

VISTE IN SQL

- Le viste sono tabelle **'virtuali'** derivate da tabelle esistenti nel db.
- Possono essere definite per nascondere dei dati da alcune tabelle (es. per questioni di privacy), per combinare più tabelle, per creare report, etc.

- Sintassi:

```
CREATE VIEW ViewName
AS SelectStatement;
```



VISTE IN SQL - ESEMPIO

```
CREATE VIEW WORKS_ON1
AS SELECT FNAME, LNAME, PNAME, HOURS
FROM EMPLOYEE, PROJECT, WORKS_ON
WHERE SSN=ESSN AND PNO=PNUMBER;
```

WORKS_ON1

FNAME	LNAME	PNAME	HOURS
-------	-------	-------	-------

VISTE IN SQL - ESEMPIO

```
CREATE VIEW WORKS_ON1
AS SELECT FNAME, LNAME, PNAME, HOURS
FROM EMPLOYEE, PROJECT, WORKS_ON
WHERE SSN=ESSN AND PNO=PNUMBER;
```

viste } materializzate
virtuali

WORKS_ON1

FNAME	LNAME	PNAME	HOURS
-------	-------	-------	-------

Trovare nome e costo medio dei pezzi che hanno almeno due fornitori (usare HAVING).